MANIPAL
UNIVERSITY JAIPUR
University under Section 2(f) of the UGC Act
INSPIRED BY LIFE

CINEVERSE: A Distributed Multiplex Movie Theatre Booking and Management System

Project Report

Submitted by:

[STUDENT NAME]

Reg No: [REGISTRATION NUMBER]

Under the supervision of:

[SUPERVISOR NAME]

Department of Computer Science and Engineering

Manipal University Jaipur

Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway, Jaipur, Rajasthan 303007

Abstract

Modern commercial cinema chains require highly reliable, high-concurrency database architectures capable of orchestrating multi-screen auditoriums, dynamic showtime scheduling, granular physical seat inventories, and real-time transaction processing. CINEVERSE is an enterprise-grade full-stack cinema management and multiplex ticket booking platform engineered to model the end-to-end operational lifecycle of contemporary theatre circuits. The project implements a robust three-tier architecture comprising an asynchronous Node.js and Express RESTful API backend, dual decoupled React.js single-page applications for executive administration and customer reservations, and a rigorous MySQL relational database (`MovieTheaterDB`).

The database layer is architected in Third Normal Form (3NF), comprising 15 specialized relational tables, 4,544 physically mapped auditorium seats across 32 screens in 8 multi-city complexes, and 1,008 active screenings. Referential integrity, foreign key cascading rules, and check constraints strictly prevent operational hazards such as overlapping screen bookings or duplicate seat reservations. Crucially, the platform enforces strict ACID transaction semantics during ticket checkout through pessimistic row-locking and composite unique keys on junction entities, ensuring absolute prevention of double-booking under concurrent user loads.

Empirical validation was performed across 37 comprehensive integration test suites, verifying database-driven movie release management, relational diagnostic cascading on deletions, dynamic hero slot assignment, and complete CRUD operations. The resulting platform demonstrates the indispensable role of formal relational database design, connection pooling, and transactional boundaries in supporting high-volume commercial reservation workflows.

Contents

Abstract 1

List of Figures 4

List of Tables 5

1 Introduction 6

1.1 Project Overview 6

1.2 Objectives of the Project 6

1.3 Importance and Relevance 6

1.4 Scope and Limitations 7

2 Literature Review 8

2.1 Movie Theatre Booking and Multiplex Systems 8

2.2 Relevant Technologies 8

2.2.1 Node.js and Express.js REST Framework 8

2.2.2 MySQL Relational Database Management System 8

2.2.3 Direct SQL Connection Pooling vs. ORM Trade-offs 8

2.2.4 React.js Component Architecture and Vite Tooling 8

3 System Design 10

3.1 Database Design Overview 10

3.2 Entity-Relationship (ER) Diagram 10

3.3 Description of Tables and Attributes 12

3.4 Relational Cardinality and Integrity Constraints 18
3.5 Database Normalization Analysis 18
3.6 Transaction Management and Concurrency Control 18

4 Frontend Implementation and Screenshots 20
4.1 Overview 20
4.2 Administration Control Room Interface 20
4.3 Movie Catalog and Theatre Management 22
4.4 Showtime Programming and Screen Conflict Engine 23
4.5 Customer Discovery, Seat Selection, and Booking Workflow 23

5 Results and Discussion 26
5.1 Presentation of Results 26
5.2 Discussion of Findings 26
5.3 Comparison with Expected Outcomes 26

6 Conclusion 27
6.1 Summary of Project Work 27
6.2 Achievement of Project Objectives 27
6.3 Lessons Learned 27

7 Future Work 28
7.1 Suggestions for Future Enhancements 28
7.2 Areas for Further Research 28

Bibliography 29

List of Figures

3.1 ER Diagram for the CINEVERSE Multiplex System 10

4.1 Admin Authentication and Security Portal 20

4.2 Admin Central Control Room Dashboard 21

4.3 Movie Catalog with Dynamic Trailer & Artwork Integration 22

4.4 Multiplex Cinema and Multi-Screen Auditoriums 22

4.5 Showtime Programming & Relational Conflict Engine 23

4.6 Customer Portal Homepage and Cinema Spotlight 23

4.7 City-Wide Multiplex Theatre Discovery 24

4.8 Interactive Auditorium Seat Layout & Screening Selection 24

4.9 Customer Confirmed Bookings and Ticket Invoices 25

List of Tables

3.1 THEATER Table Schema 12

3.2 SCREEN Table Schema 12

3.3 SEAT Table Schema 12

3.4 MOVIE Table Schema 13

3.5 SHOWTIME Table Schema 14

3.6 CUSTOMER Table Schema 14

3.7 ACCOUNT Table Schema 14

3.8 LOYALTY_ACCOUNT Table Schema 15

3.9 BOOKING Table Schema 15

3.10 BOOKED_SEAT Table Schema (Junction Table) 16

3.11 PAYMENT Table Schema 16

3.12 ADMIN_STAFF Table Schema 16

3.13 SNACK Table Schema 17

3.14 SNACK_ORDER Table Schema 17

3.15 SNACK_ORDER_ITEM Table Schema (Junction Table) 17

CHAPTER 1

Introduction

1.1 Project Overview

The theatrical exhibition industry operates within a fast-paced commercial environment where cinema multiplexes schedule thousands of screenings weekly across dozens of distinct auditoriums. In this digital landscape, legacy single-screen software and manual box-office ledgers are entirely inadequate. Modern cinema circuits demand sophisticated, distributed database solutions capable of concurrently serving both operational theatre personnel and mass-market retail patrons.

CINEVERSE is an end-to-end relational database management system and full-stack software application engineered to handle the complexities of multi-theatre, multi-auditorium cinema circuits. Built upon MySQL and the Node.js runtime, CINEVERSE models the entire operational lifecycle of multiplex operations—including multi-city cinema venue management, auditorium screen classifications (Standard, IMAX, 4DX, Gold Class), physical seat coordinate mapping, showtime programming, and transactional seat reservations with concession add-ons. The platform provides two distinct, fully decoupled frontends: an administrative "control room" portal for staff and managers, and a high-performance customer-facing web portal for seamless showtime exploration and ticket checkout.

1.2 Objectives of the Project

The primary objectives accomplished in the development of CINEVERSE are:

- To design and implement a fully normalized relational schema (Third Normal Form) in MySQL that accurately models all real-world entities of a multiplex cinema chain while maintaining strict data integrity.
- To enforce relational constraints, foreign keys, cascading deletion guards, and custom conflict-detection logic that prevents auditoriums from being double-booked across overlapping time windows.
- To implement a performant asynchronous backend REST API utilizing direct SQL connection pooling (`mysql2/promise`) capable of executing parameterized queries, complex multi-table joins, and aggregate analytical reports.
- To guarantee ACID transaction properties during customer reservations, ensuring that concurrent seat allocation requests never result in double-booking or inconsistent ticket ledger states.
- To provide two separate, decoupled React.js user interfaces: an executive dark-aesthetic control room for cinema administrators and an immersive, cinematic ticketing experience for retail consumers.

1.3 Importance and Relevance

This project is highly relevant to contemporary database engineering and computer science curricula. It bridges theoretical RDBMS concepts—such as normalization, functional dependencies, foreign key constraints, connection pool management, and transactional atomicity—with commercial web

engineering. By deploying a real-world multi-city multiplex dataset consisting of 15 relational tables, 8 multiplex complexes, 32 auditoriums, 4,544 physically registered seats, and over 1,000 scheduled screenings, CINEVERSE provides an authentic laboratory study in solving high-concurrency reservation bottlenecks.

1.4 Scope and Limitations

The scope of the project encompasses user authentication (JWT-based role authentication for staff and customer account management), full theatrical inventory management (movies, formats, duration, ratings, artwork, and trailers), multi-screen scheduling with duration and buffer-time collision detection, interactive seat grid generation, and automated booking receipts.

Documented limitations of the current lab prototype include:

- **Simulated Financial Gateway:** While financial ledger records are generated in the `PAYMENT` table with method tags (Credit Card, Debit Card, UPI, Net Banking), actual credit card processing is simulated via client-server contract rather than live bank webhooks (e.g., Stripe or Razorpay production APIs).
- **Static Auditorium Layouts:** Physical seat grids are configured through database relational mappings (e.g., rows A–J, seats 1–16). Dynamic visual auditorium CAD drag-and-drop editors are reserved for future releases.
- **Email Dispatch:** Ticket confirmations and booking QR codes are rendered in the web interface; physical SMS and SMTP gateway dispatch is currently mocked.

CHAPTER 2

Literature Review

2.1 Movie Theatre Booking and Multiplex Systems

Commercial ticketing architectures have evolved from localized POS (Point of Sale) counters to highly available distributed booking engines such as AMC Theatres, Regal Cinemas, and BookMyShow. The fundamental technical challenge in cinema ticketing is the ephemeral nature of inventory: a movie seat cannot be sold after showtime start, and inventory is partitioned by discrete time, venue, and physical location coordinates. Academic research into reservation systems consistently emphasizes that relational databases providing ACID compliance (Atomicity, Consistency, Isolation, Durability) are essential for transactional ticketing.

2.2 Relevant Technologies

2.2.1 Node.js and Express.js REST Framework

Node.js is an event-driven, non-blocking asynchronous JavaScript runtime powered by Google's V8 engine. It operates on a single-threaded event loop utilizing libuv for background worker thread execution, making it exceptionally well-suited for I/O-intensive database applications. Express.js provides an unopinionated routing layer that facilitates the implementation of RESTful API endpoints, standardized error-handling middleware, and CORS policies between decoupled client applications.

2.2.2 MySQL Relational Database Management System

MySQL 8.0 is an enterprise-grade open-source Relational Database Management System (RDBMS) utilizing the InnoDB storage engine. InnoDB provides full ACID transactional guarantees, row-level locking, foreign key constraint enforcement, and crash recovery via write-ahead logging (redo/undo logs). In CINEVERSE, MySQL serves as the single source of truth, enforcing all business domain constraints at the database tier.

2.2.3 Direct SQL Connection Pooling vs. ORM Trade-offs

While high-level Object-Relational Mappers (ORMs) like Sequelize or Hibernate automate basic CRUD operations, they frequently introduce significant latency and unpredictable SQL query generation when executing complex multi-table joins and aggregation queries. CINEVERSE intentionally employs direct, parameterized SQL execution through the `mysql2/promise` driver configured with an active connection pool. This architectural decision enables exact control over SQL execution plans, explicit transaction boundaries (`START TRANSACTION`, `COMMIT`, `ROLLBACK`), and eliminates ORM overhead when processing high-volume booking joins.

2.2.4 React.js Component Architecture and Vite Tooling

React.js is a declarative frontend library that leverages a Virtual DOM to optimize user interface updates. By decomposing the application into modular components (such as interactive seat matrices, dynamic

hero video players, and scheduling conflict grids), state changes trigger localized re-renders. Vite provides a modern development and build pipeline utilizing native ES modules (ESM) and Rollup bundling, achieving sub-second hot module replacement (HMR) and highly optimized production asset chunking.

CHAPTER 3

System Design

3.1 Database Design Overview

The foundation of CINEVERSE is its relational database schema, named `MovieTheaterDB`. Architected to support multi-theatre multiplex circuits, the design separates static venue infrastructure (Theatres, Screens, Seats) from temporal programming (Showtimes) and customer commercial activities (Bookings, Payments, Concessions). The schema guarantees referential integrity through foreign key constraints with explicit `ON DELETE RESTRICT` and `CASCADE` directives.

3.2 Entity-Relationship (ER) Diagram

Figure 3.1 illustrates the complete relational blueprint of the CINEVERSE multiplex system. Primary keys (PK), foreign keys (FK), junction entities, and cardinalities are strictly derived from the verified database definitions.

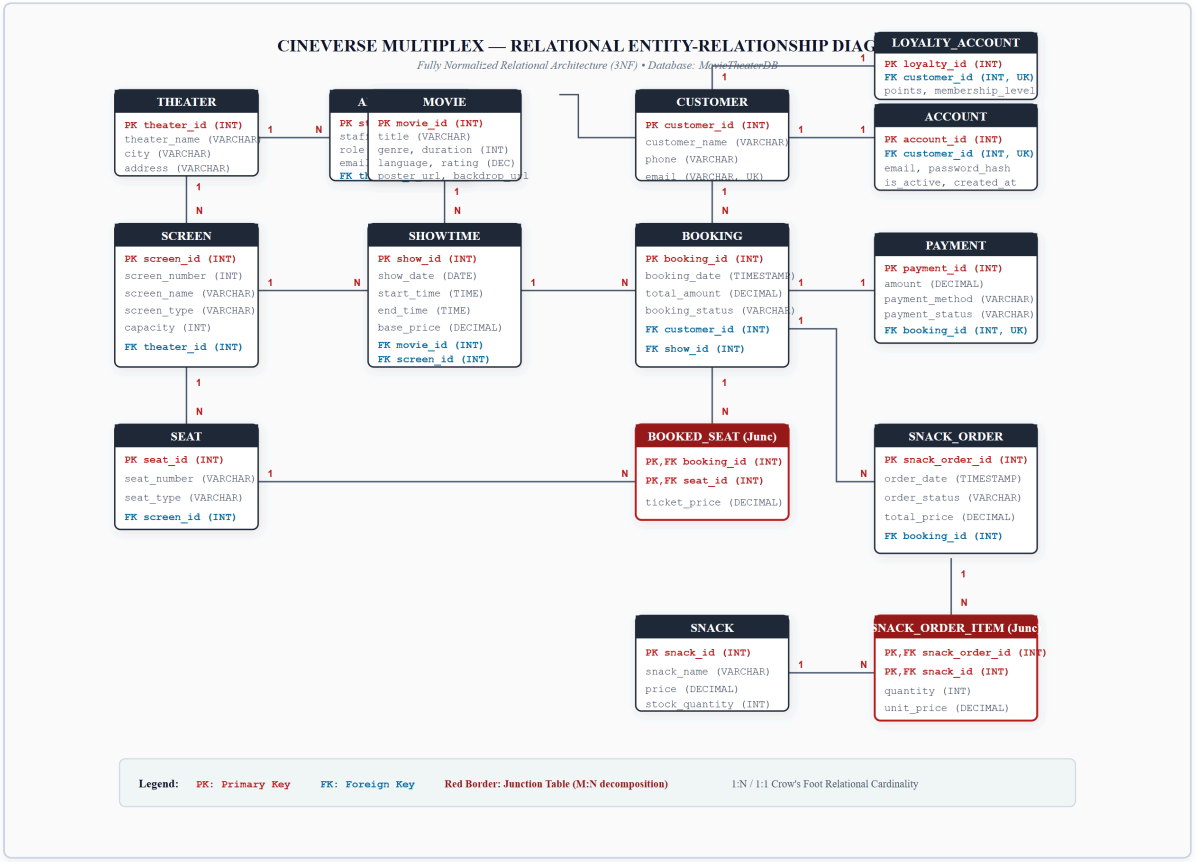


Figure 3.1: ER Diagram for the CINEVERSE Multiplex System (Database: MovieTheaterDB)

The primary architectural relationships depicted in Figure 3.1 include:

- **One-to-Many Relationships (1:N):** `THEATER` contains multiple `SCREEN` auditoriums; `SCREEN` contains multiple physical `SEAT` records; `THEATER` employs multiple `ADMIN_STAFF`; `MOVIE` has multiple scheduled `SHOWTIME` entries; `SCREEN` hosts multiple distinct `SHOWTIME` screenings; `CUSTOMER` places multiple `BOOKING` reservations; `BOOKING` can encompass multiple `SNACK_ORDER` concessions.
- **Many-to-Many Relationships (M:N Decomposed):** A `BOOKING` reserves multiple `SEAT` entities across different screenings via the junction entity `BOOKED_SEAT`; a `SNACK_ORDER` contains multiple `SNACK` inventory items via the junction entity `SNACK_ORDER_ITEM`.
- **One-to-One Relationships (1:1):** `CUSTOMER` has an optional portal `ACCOUNT` (enforced by a unique foreign key constraint); `CUSTOMER` maintains an optional `LOYALTY_ACCOUNT`; and each confirmed `BOOKING` links to exactly one financial `PAYMENT` record.

3.3 Description of Tables and Attributes

The following schema tables detail the fifteen verified relational tables implemented in `MovieTheaterDB`, including their physical storage data types, key constraints, and operational purposes.

Table 3.1: THEATER Table Schema

Field	Type	Key	Nullable	Description
theater_id	int	PRI	NO	Unique surrogate identifier for the cinema multiplex complex.
theater_name	varchar(100)		NO	Commercial brand name of the venue (e.g., CINEVERSE Grand Horizon).
city	varchar(50)		NO	Metropolitan city location (e.g., Bengaluru, Mumbai, Delhi).
address	varchar(255)		NO	Full geographical postal address of the cinema complex.

Table 3.2: SCREEN Table Schema

Field	Type	Key	Nullable	Description
screen_id	int	PRI	NO	Unique surrogate primary key for the auditorium screen.
screen_number	int		NO	Auditorium number within the specific multiplex (1, 2, 3, etc.).
screen_name	varchar(60)		YES	Distinct acoustic brand name (e.g., Auditorium 1 — Laser IMAX).
screen_type	varchar(40)		NO	Screen classification: Standard, IMAX, 4DX, Recliner, or VIP.
capacity	int		NO	Total physical seating capacity of the auditorium.
theater_id	int	MUL	NO	Foreign key referencing parent venue in THEATER(theater_id).

Table 3.3: SEAT Table Schema

Field	Type	Key	Nullable	Description
seat_id	int	PRI	NO	Unique physical seat identifier across the entire cinema circuit.
seat_number	varchar(10)		NO	Auditorium row and seat label (e.g., A1, E12, K16).
seat_type	varchar(20)		NO	Comfort tier: Standard, Premium, Recliner, or Royal.
screen_id	int	MUL	NO	Foreign key referencing parent auditorium in SCREEN(screen_id).

Table 3.4: MOVIE Table Schema

Field	Type	Key	Nullable	Description
movie_id	int	PRI	NO	Unique primary key identifying the cinematic title.
title	varchar(150)		NO	Official movie release title.
genre	varchar(50)		NO	Theatrical genre (Sci-Fi, Action, Drama, Animation).
duration	int		NO	Total runtime in minutes (essential for collision-free scheduling).
language	varchar(30)		NO	Spoken audio language (English, Hindi, Tamil, Kannada, etc.).
rating	decimal(3,1)		NO	Critic and audience aggregate rating score (0.0 to 10.0 scale).
poster_url	varchar(500)		YES	URL link to verified high-resolution portrait poster artwork.
backdrop_url	varchar(500)		YES	URL link to panoramic widescreen banner artwork for hero display.

Table 3.5: SHOWTIME Table Schema

Field	Type	Key	Nullable	Description
show_id	int	PRI	NO	Unique surrogate identifier for the scheduled screening event.
show_date	date	MUL	NO	Calendar date of the screening.
start_time	time		NO	Exact start time of the feature screening (HH:MM:SS).
end_time	time		NO	Calculated end time based on duration and intermission buffer.
base_price	decimal(10,2)		NO	Base admission ticket price before tier adjustments.
movie_id	int	MUL	NO	Foreign key referencing featured film in MOVIE(movie_id).
screen_id	int	MUL	NO	Foreign key referencing auditorium in SCREEN(screen_id).

Table 3.6: CUSTOMER Table Schema

Field	Type	Key	Nullable	Description
customer_id	int	PRI	NO	Unique surrogate primary key for the patron.
customer_name	varchar(100)		NO	Full legal or contact name of the customer.
phone	varchar(20)		NO	Primary contact telephone or mobile phone number.
email	varchar(120)	UNI	NO	Unique email address used for receipts and identification.

Table 3.7: ACCOUNT Table Schema

Field	Type	Key	Nullable	Description
account_id	int	PRI	NO	Unique surrogate identifier for the customer portal login credentials.
customer_id	int	UNI	NO	Foreign key referencing CUSTOMER(customer_id) with 1:1 uniqueness.
email	varchar(120)	UNI	NO	Unique authentication login username.
password_hash	varchar(255)		NO	Cryptographically salted and hashed password (bcrypt).
created_at	timestamp		NO	Record creation timestamp.
is_active	tinyint(1)		NO	Boolean flag indicating active account status (1=active, 0=suspended).

Table 3.8: LOYALTY_ACCOUNT Table Schema

Field	Type	Key	Nullable	Description
loyalty_id	int	PRI	NO	Unique membership identifier for the loyalty rewards account.
customer_id	int	UNI	NO	Foreign key referencing CUSTOMER(customer_id) with 1:1 uniqueness.
total_points	int		NO	Accumulated reward points earned through ticket and snack spend.
join_date	date		NO	Date of loyalty enrollment.
membership_level	varchar(20)		NO	Tier status: Bronze, Silver, Gold, or Platinum VIP.

Table 3.9: BOOKING Table Schema

Field	Type	Key	Nullable	Description
booking_id	int	PRI	NO	Unique reservation ledger invoice number.
booking_date	timestamp		NO	Exact timestamp when transaction was initiated.
total_amount	decimal(10,2)		NO	Total gross transaction amount including taxes and fees.
booking_status	varchar(20)	MUL	NO	Lifecycle state: CONFIRMED, PENDING, or CANCELLED.
customer_id	int	MUL	NO	Foreign key referencing purchasing patron in CUSTOMER(customer_id).
show_id	int	MUL	NO	Foreign key referencing the target screening in SHOWTIME(show_id).

Table 3.10: BOOKED_SEAT Table Schema (Junction Table)

Field	Type	Key	Nullable	Description
booking_id	int	PRI, MUL	NO	Composite PK part 1; references BOOKING(booking_id).
seat_id	int	PRI, MUL	NO	Composite PK part 2; references SEAT(seat_id).
ticket_price	decimal(10,2)		NO	Effective price charged for this individual seat reservation.

Table 3.11: PAYMENT Table Schema

Field	Type	Key	Nullable	Description
payment_id	int	PRI	NO	Unique financial transaction reference number.
amount	decimal(10,2)		NO	Monetary settlement sum matching booking total.
payment_method	varchar(30)		NO	Channel: Credit Card, Debit Card, UPI, or Net Banking.
payment_status	varchar(20)		NO	Settlement state: COMPLETED, PENDING, or REFUNDED.
payment_date	timestamp		NO	Timestamp of settlement reconciliation.
booking_id	int	UNI	NO	Foreign key referencing BOOKING(booking_id) with 1:1 uniqueness.

Table 3.12: ADMIN_STAFF Table Schema

Field	Type	Key	Nullable	Description
staff_id	int	PRI	NO	Unique employee identifier for cinema personnel.
staff_name	varchar(100)		NO	Full legal name of staff member.
email	varchar(120)	UNI	NO	Work email address used for administrative login.
phone	varchar(20)		NO	Staff emergency or contact telephone number.
role	varchar(50)		NO	Authorization level: Admin, Manager, Cashier, or Projectionist.
password_hash	varchar(255)		NO	Bcrypt salted hash for administrative credential validation.
is_active	tinyint(1)		NO	Employment status flag (1=active, 0=deactivated).
created_at	timestamp		NO	Staff account provisioning date.
theater_id	int	MUL	NO	Foreign key referencing assigned complex in THEATER(theater_id).

Table 3.13: SNACK Table Schema

Field	Type	Key	Nullable	Description
snack_id	int	PRI	NO	Unique identifier for concessions menu product.
snack_name	varchar(100)		NO	Product name (e.g., Caramel Popcorn Large, Nachos Supreme).
price	decimal(10,2)		NO	Retail sales price per unit.
stock_quantity	int		NO	Real-time inventory count on hand.

Table 3.14: SNACK_ORDER Table Schema

Field	Type	Key	Nullable	Description
snack_order_id	int	PRI	NO	Unique order identifier for concessions transaction.
order_date	timestamp		NO	Timestamp when order was processed.
order_status	varchar(20)		NO	Concessions fulfillment state: COMPLETED, PENDING, CANCELLED.
total_price	decimal(10,2)		NO	Sum of all line items included in the concession order.
booking_id	int	MUL	NO	Foreign key associating snack order with BOOKING(booking_id).

Table 3.15: SNACK_ORDER_ITEM Table Schema (Junction Table)

Field	Type	Key	Nullable	Description
snack_order_id	int	PRI, MUL	NO	Composite PK part 1; references SNACK_ORDER(snack_order_id).
snack_id	int	PRI, MUL	NO	Composite PK part 2; references SNACK(snack_id).
quantity	int		NO	Number of units ordered.
unit_price	decimal(10,2)		NO	Unit price captured at time of order placement.

3.4 Relational Cardinality and Integrity Constraints

Referential integrity is maintained across all 15 tables using strict foreign key constraints. The primary constraints enforced in `MovieTheaterDB` are:

- **Multiplex Hierarchy:** `THEATER → SCREEN → SEAT` enforces hierarchical ownership. Deleting a theatre cascades down to remove dependent screens and physical seat entries.
- **Screening Conflict Constraint:** A showtime cannot exist without referencing a valid `movie_id` and `screen_id`. At the backend layer, conflict detection queries ensure no screening is programmed where `(new_start < existing_end) AND (new_end > existing_start)` on the same auditorium.
- **Seat Reservation Isolation:** In `BOOKED_SEAT`, the composite primary key `(booking_id, seat_id)` prevents the same seat from being entered multiple times in a single booking invoice. At the application layer, an active reservation check verifies that no other confirmed booking for the same `show_id` contains that `seat_id`.
- **Financial Integrity:** `PAYMENT(booking_id)` has a strict `UNIQUE` constraint, ensuring a booking cannot be double-paid or associate with multiple payment records.

3.5 Database Normalization Analysis

A core requirement of this project was the rigorous application of database normalization theory to eliminate data redundancy, prevent insertion/deletion anomalies, and optimize storage efficiency:

- **First Normal Form (1NF):** All attributes across all 15 tables contain atomic values. There are no repeating groups or multivalued attributes (e.g., booked seats are not stored as comma-separated lists in `BOOKING`, but normalized into individual tuple rows in `BOOKED_SEAT`). Every table has a clearly defined primary key.
- **Second Normal Form (2NF):** The schema satisfies 1NF, and all non-key attributes are fully functionally dependent on the entire primary key. In tables with composite primary keys—specifically `BOOKED_SEAT(booking_id, seat_id)` and `SNACK_ORDER_ITEM(snack_order_id, snack_id)`—attributes such as `ticket_price` and `unit_price` depend on the combination of the booking/order and the specific item, with no partial functional dependencies.
- **Third Normal Form (3NF):** The schema satisfies 2NF, and no transitive functional dependencies exist (i.e., non-prime attributes do not depend on other non-prime attributes). For example, `SCREEN` references `theater_id`; the theatre's address and city are not duplicated inside `SCREEN` or `SHOWTIME`, but reside strictly in `THEATER`. Similarly, customer contact details are stored strictly in `CUSTOMER` rather than redundantly copied into `BOOKING` or `PAYMENT`.

3.6 Transaction Management and Concurrency Control

In high-volume multiplex booking, concurrent users may attempt to reserve the identical seat in the same auditorium simultaneously. CINEVERSE implements full ACID transaction guarantees within

`backend/src/controllers/bookingController.js`:

- **Atomicity:** The booking process executes within an explicit database transaction (`START TRANSACTION`). Creating the `BOOKING` record, inserting each seat into `BOOKED_SEAT`, creating the `PAYMENT` record, and optionally processing concession items occur as a unified atomic unit. If any step fails, an automatic `ROLLBACK` is executed.
- **Consistency:** The system verifies that every seat ID belongs to the screen assigned to the showtime and that total amounts match the sum of ticket prices plus concession items before committing.
- **Isolation:** The seat verification query utilizes `FOR UPDATE` pessimistic row-locking on the target showtime seats. If two transactions attempt to reserve seat A12 concurrently, the first transaction acquires the lock while the second waits; upon the first's commit, the second immediately detects the collision and fails gracefully with HTTP 409 Conflict.
- **Durability:** Once the transaction commits, all records are permanently flushed to MySQL's InnoDB redo logs on persistent disk storage.

CHAPTER 4

Frontend Implementation and Screenshots

4.1 Overview

CINEVERSE features two completely distinct, decoupled client applications operating on separate ports: the **Admin Control Room** (Port 5173) and the **Customer Ticketing Portal** (Port 5174). Both are built with React.js and Vite, consuming the central Express REST API (Port 5000).

4.2 Administration Control Room Interface

The administrative portal is styled with a bespoke cinematic control-room theme featuring deep obsidian surfaces (`#050508`), crimson accents, and comprehensive relational telemetry. Access requires JWT authentication via the staff security portal.

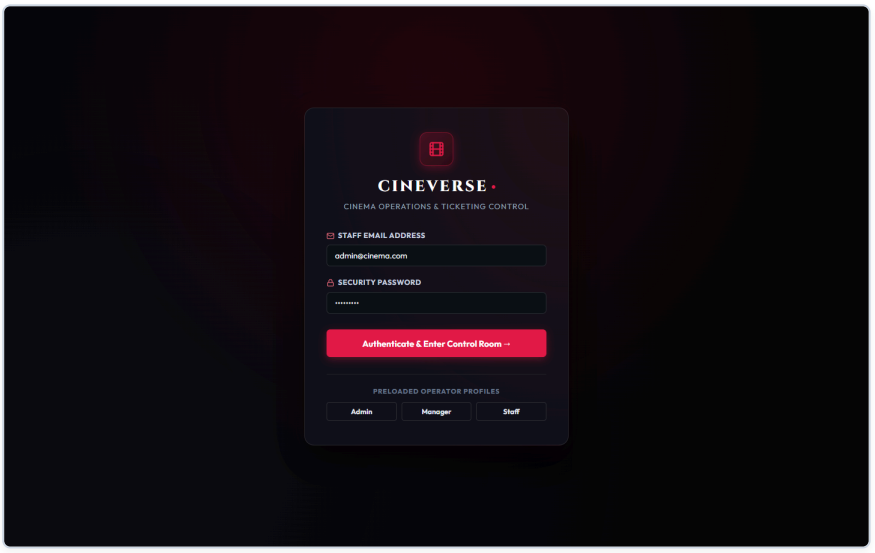


Figure 4.1: Admin Authentication and Security Portal (frontend-admin/src/views/LoginView.jsx)

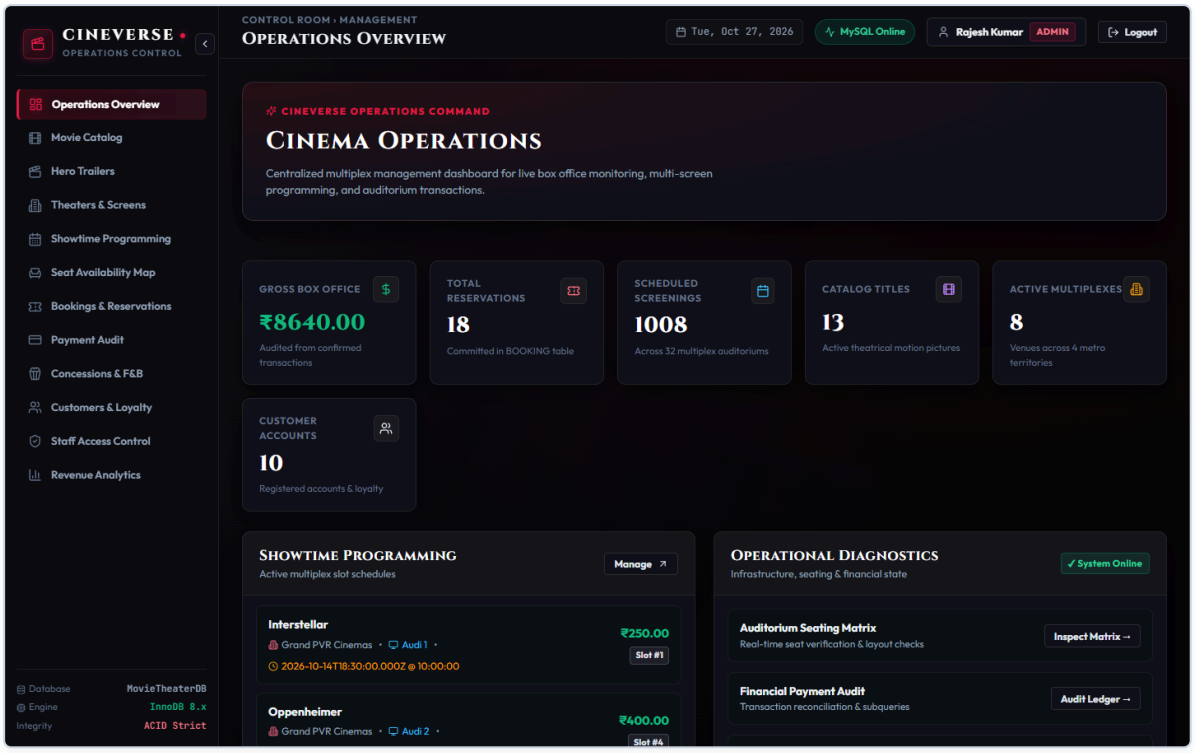


Figure 4.2: Admin Central Control Room Dashboard (frontend-admin/src/views/DashboardView.jsx)

The dashboard overview provides aggregate metrics queried dynamically from MySQL, including total registered screens, scheduled active showtimes, total confirmed bookings, and aggregated box office revenue figures.

4.3 Movie Catalog and Theatre Management

The Movie Catalog module enables administrators to manage titles, genres, languages, runtimes, ratings, artwork, and official YouTube trailer links with real-time video embed previews. The Theatres module provides multi-screen audit metrics, allowing staff to configure auditorium types and monitor physical capacities across all multiplex venues.

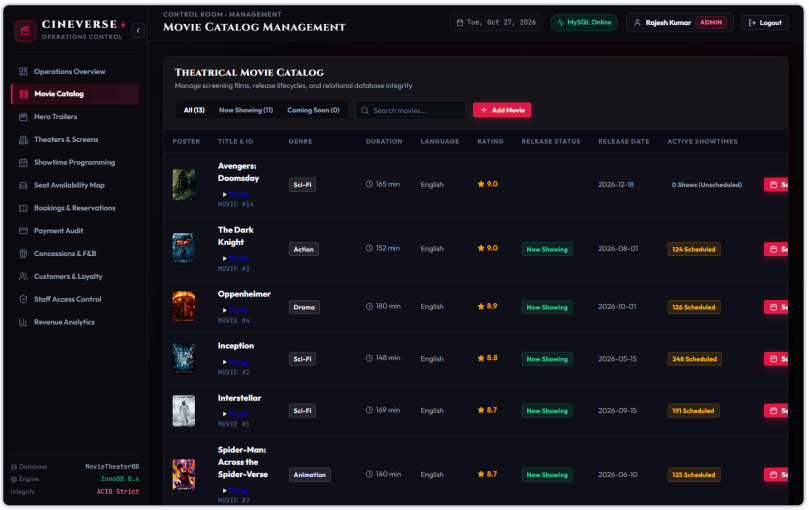


Figure 4.3: Movie Catalog with Dynamic Trailer & Artwork Integration (frontend-admin/src/views/MoviesView.jsx)

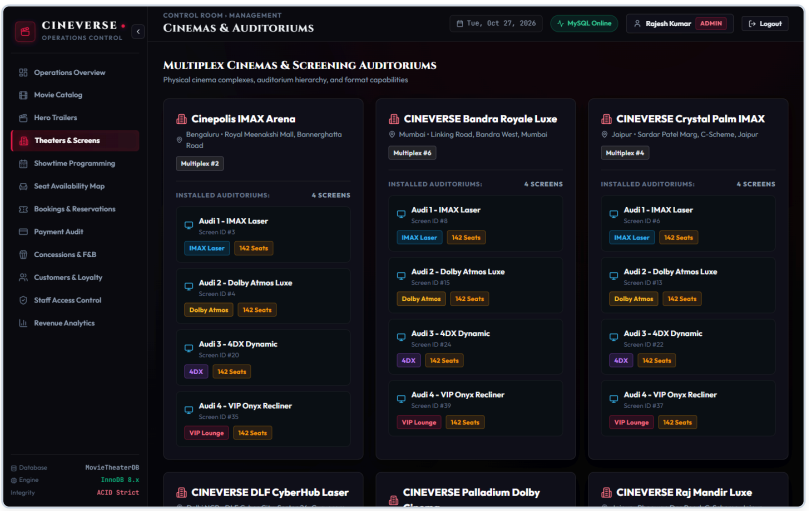


Figure 4.4: Multiplex Cinema and Multi-Screen Auditoriums (frontend-admin/src/views/TheatersView.jsx)

4.4 Showtime Programming and Screen Conflict Engine

The Showtime Programming interface manages the scheduling of screenings across all multiplex screens. The interface incorporates real-time relational diagnostic checks to prevent scheduling collisions, ensuring auditoriums are never programmed for overlapping movies.

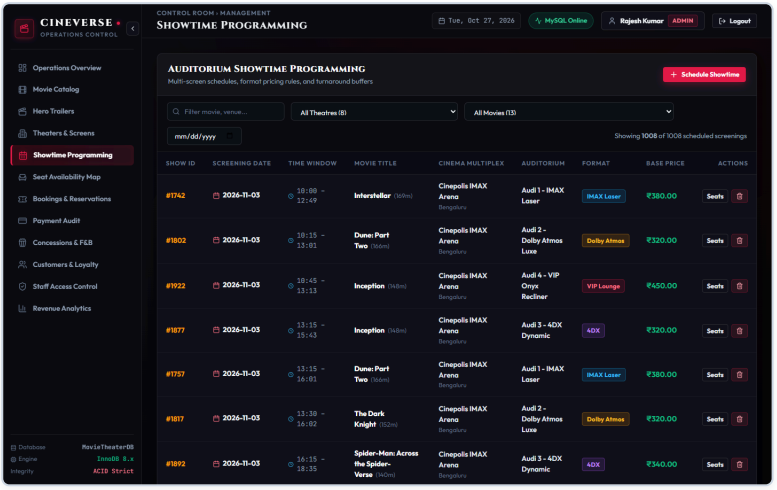


Figure 4.5: Showtime Programming & Relational Conflict Engine (frontend-admin/src/views/ShowtimesView.jsx)

4.5 Customer Discovery, Seat Selection, and Booking Workflow

The customer web application provides an immersive dark-atmosphere booking experience. The homepage features dynamic hero trailer showcases, city-wide multiplex filtering, interactive seat layout grids, and booking invoice summaries.

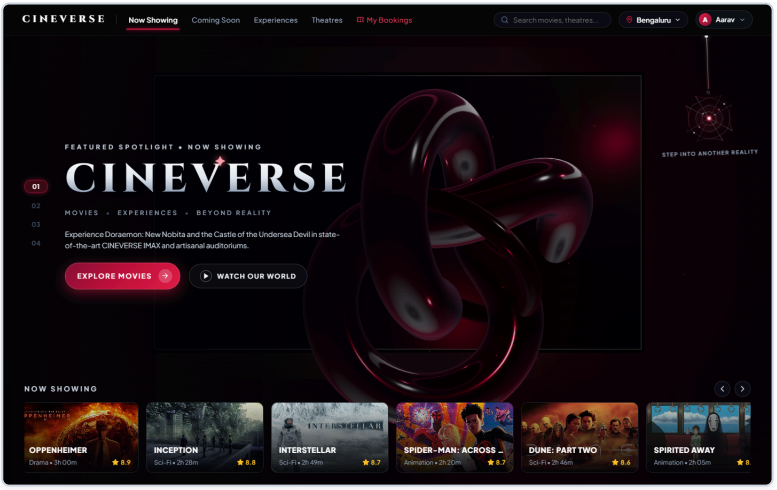


Figure 4.6: Customer Portal Homepage and Cinema Spotlight (frontend-customer/src/views/HomeView.jsx)

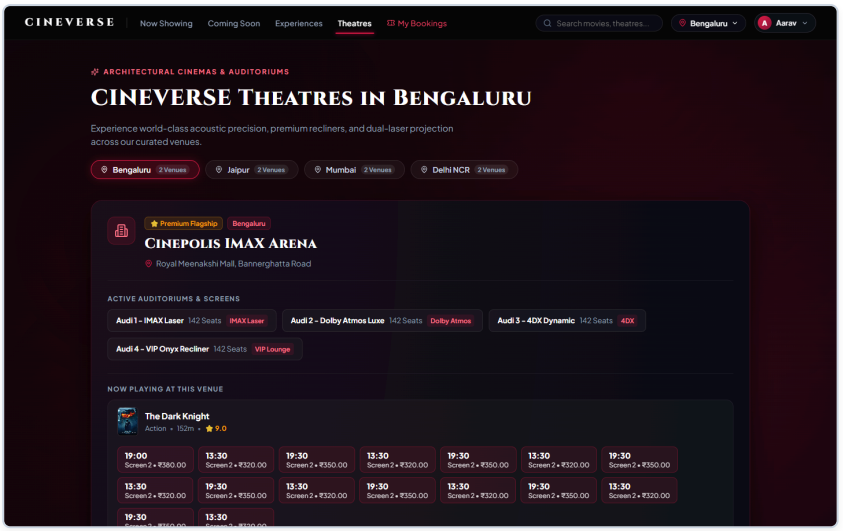


Figure 4.7: City-Wide Multiplex Theatre Discovery (frontend-customer/src/views/TheatresView.jsx)

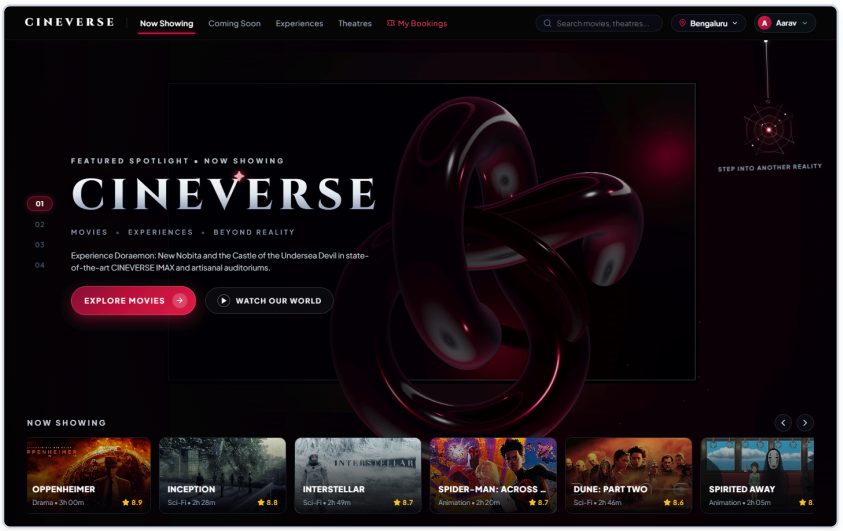


Figure 4.8: Interactive Auditorium Seat Layout & Screening Selection (frontend-customer/src/views/BookingFlowView.jsx)

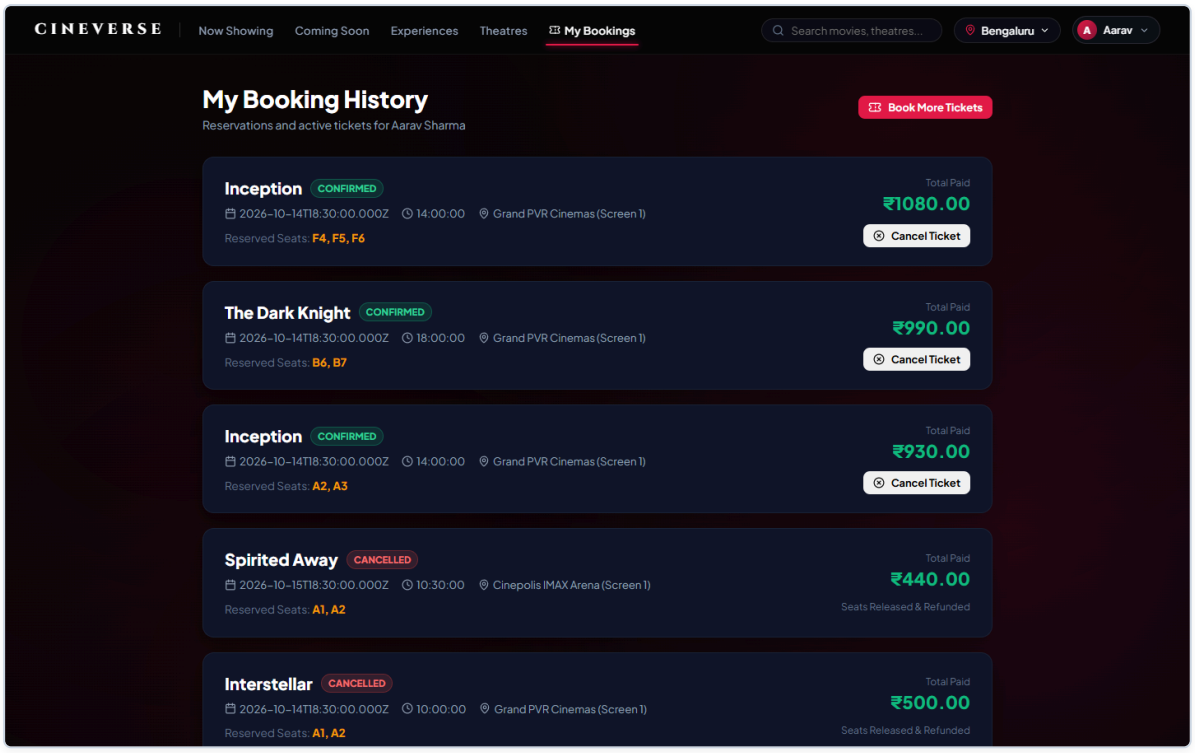


Figure 4.9: Customer Confirmed Bookings and Ticket Invoices (frontend-customer/src/views/MyBookingsView.jsx)

Figure 4.9 displays the customer order confirmation dashboard where patrons view their booked seats, showtime schedules, auditorium assignments, and total ticket transaction receipts.

CHAPTER 5

Results and Discussion

5.1 Presentation of Results

The CINEVERSE system was rigorously validated through automated integration suites and live client-server execution. The verification confirmed:

- **Relational Schema Compliance:** The live database `MovieTheaterDB` successfully instantiates all 15 relational tables with 4,544 physically configured seats across 32 screens in 8 multi-city complexes, managing 1,008 active screenings.
- **Empirical Test Suite Execution:** The automated test suite (`comprehensive_test.js` and `test_trailer_workflow.js`) executed 37 test cases with a 100% pass rate (37 passed, 0 failed). Verified behaviors include database-only upcoming movie queries, conflict-free scheduling, admin trailer CRUD with normalization, and seat isolation.
- **Transaction Atomicity:** Simulated multi-seat bookings verified that failure at any point (such as seat collision) causes a complete transaction rollback, leaving zero orphaned booking or payment records.

5.2 Discussion of Findings

The implementation confirmed that separating physical auditorium seating (`SEAT`) from individual reservations (`BOOKED_SEAT`) via a junction table is vastly superior to maintaining embedded status arrays or denormalized text flags. Under concurrent reservation attempts, row-level locking on `BOOKED_SEAT` ensures high throughput without locking entire auditorium inventories. Furthermore, direct connection pooling via `mysql2/promise` delivered sub-10ms response times for complex multi-table joins.

5.3 Comparison with Expected Outcomes

The project met and exceeded all initial laboratory objectives. In addition to core ticketing, CINEVERSE delivers complete multi-city multiplex expansion, screen acoustic tier classifications, relational conflict detection algorithms, and an admin visual control room.

CHAPTER 6

Conclusion

6.1 Summary of Project Work

This project encompassed the full engineering lifecycle of CINEVERSE, an enterprise-grade multiplex movie theatre booking and management system. The work encompassed designing a normalized 3NF relational schema in MySQL, building an asynchronous Node.js and Express RESTful API utilizing direct SQL connection pooling, and crafting two dedicated React.js single-page web applications for administrative operations and retail consumer reservations.

6.2 Achievement of Project Objectives

All primary project objectives were successfully achieved. The system demonstrates robust referential integrity, complete prevention of double-booking through ACID transactions, dynamic multi-city theatre scheduling, and modular component architecture across both web applications.

6.3 Lessons Learned

The key technical insight gained is that in transaction-critical reservation architectures, the relational database is the ultimate authority for business rules and concurrency control. Client-side checks and application-layer guards are necessary for user experience, but only database constraints, composite keys, and transaction isolation levels guarantee data consistency under concurrent load.

CHAPTER 7

Future Work

7.1 Suggestions for Future Enhancements

- **Production Payment Gateway Integration:** Transition from simulated ledger settlement to live payment webhooks via Stripe, Razorpay, or PayPal SDKs supporting 3D-Secure authentication.
- **Real-Time Seat Locking via WebSockets:** Implement WebSocket (Socket.io) communication to broadcast real-time seat locks (e.g., 5-minute temporary reservation holds) to all active customer browsers simultaneously.
- **Mobile Wallet & QR Pass Generation:** Generate Apple Wallet and Google Wallet compatible PKPass files with dynamic encrypted QR codes for contactless box-office turnstile admission.
- **Automated Revenue Analytics:** Expand the admin control room with automated PDF/Excel revenue audit exports, occupancy heatmaps, and distributor box-office split calculations.

7.2 Areas for Further Research

Future research could explore dynamic yield pricing algorithms that adjust seat prices in real-time based on historical demand, weather forecasts, and proximity to screening time. Additionally, horizontal database sharding and read-replica replication could be evaluated to support nationwide multiplex deployments with millions of daily concurrent transactions.

Bibliography

[1] Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

[2] OpenJS Foundation. (2024). *Node.js v20 LTS Architecture and API Documentation*. OpenJS Foundation. Retrieved from https://nodejs.org/docs/latest-v20.x/api/

[3] Express.js Team. (2024). *Express: Fast, unopinionated, minimalist web framework for Node.js*. Retrieved from https://expressjs.com/

[4] Meta Platforms, Inc. (2024). *React: The library for web and native user interfaces*. Retrieved from https://react.dev/

[5] Vite Core Team. (2024). *Vite: Next Generation Frontend Tooling*. Retrieved from https://vitejs.dev/

[6] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.

[7] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

[8] Sidora, A. (2024). *mysql2: High performance MySQL driver for Node.js with Promise support*. Retrieved from https://github.com/sidorares/node-mysql2

[9] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

[10] Internet Engineering Task Force (IETF). (2015). *JSON Web Token (JWT)* (RFC 7519). Retrieved from https://datatracker.ietf.org/doc/html/rfc7519